WFLOW-09: Security, Governance & Monitoring

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 9 of 10 | **Created:** January 2026 | **Last Updated:** 08/12/2026

Production Best Practices

This final notebook covers workflow security, governance, observability, and operational best practices for running workflows in production.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:admin` for governance setup
Prior Knowledge	**WFLOW-01** through **WFLOW-08**

1. Security Best Practices

Secure Workflow Design

Principle	Implementation
Least privilege	Connections with minimum required permissions
No hardcoded secrets	Use secrets vault, never inline credentials
Input validation	Sanitize all dynamic inputs
Audit logging	Log all actions for compliance
Fail secure	Default to safe state on errors

Input Sanitization

```
export default async function({ event }) {
  // NEVER do this - injection risk
  // const query = `fetch logs | filter content == "${event().title}"`;

  // SAFE: Validate and sanitize inputs
  const title = event().title;

  // Remove potentially dangerous characters
  const sanitized = title
    .replace(/["'\\]/g, '')  // Remove quotes and backslashes
    .substring(0, 200);        // Limit length

  // Use parameterized queries when possible
  return { sanitized_title: sanitized };
}
```

Secure HTTP Requests

```
export default async function({ event, env }) {
  // NEVER log or expose secrets
  // console.log(env.API_TOKEN);  // BAD!

  // ALWAYS use HTTPS
  const response = await fetch('https://api.example.com/endpoint', {  // NOT
http://
    headers: {
      'Authorization': `Bearer ${env.API_TOKEN}`  // Token from secrets
    }
  });

  // DON'T return sensitive data
  return {
    status: response.status,
    success: response.ok
    // NOT: response_body: await response.text()  // May contain secrets
  };
}
```

2. Secrets Management

Using Environment Secrets

Secrets are stored securely and accessed via `env.SECRET_NAME`.

Creating Secrets:

1. Open workflow editor
2. Go to **Settings** → **Secrets**
3. Add secret with name and value
4. Reference as `{{ env.SECRET_NAME }}`

Secret Naming Convention

Pattern	Example	Use
`<SERVICE>_API_TOKEN`	`SLACK_API_TOKEN`	API authentication
`<SERVICE>_WEBHOOK_URL`	`PAGERDUTY_WEBHOOK_URL`	Webhook endpoints
`<SERVICE>_<ENV>_CRED`	`SERVICENOW_PROD_CRED`	Environment-specific

Secrets in JavaScript

```
export default async function({ env }) {
  // Access secrets via env object
  const apiToken = env.EXTERNAL_API_TOKEN;
  const webhookUrl = env.SLACK_WEBHOOK_URL;

  // Secrets are never logged or exposed
  if (!apiToken) {
    throw new Error('Missing required secret: EXTERNAL_API_TOKEN');
  }

  return { configured: true };
}
```

Secrets in YAML

```
input:
  url: "https://api.example.com/webhook"
  headers:
    Authorization: "Bearer {{ env.API_TOKEN }}"
    X-Api-Key: "{{ env.API_KEY }}"
```

Secret Rotation

1. Create new secret with updated value
2. Update workflow to use new secret
3. Test workflow execution
4. Remove old secret

3. Setting Up Third-Party Connections

WFLOW-03 introduces the Dynatrace **Connection** abstraction; WFLOW-05 walks through PagerDuty and ServiceNow workflow tasks. This section closes the remaining gap: **how to mint the actual vendor-side credentials** that those Connections wrap, with current vendor scope/auth model details and rotation patterns that don't break running workflows.

3.1 Slack — App + OAuth Token

The Dynatrace Slack action authenticates as a Slack app (bot token), not as a user. Steps:

1. **Create the app** at `https://api.slack.com/apps` → **From scratch** → name it (e.g. *Dynatrace Alerts*) and pick the target workspace.
2. **Add OAuth scopes** under **OAuth & Permissions** → **Bot Token Scopes**:
   - `chat:write` — *"Send messages as your Slack app"* (works with both Bot and User token types; replaces legacy `chat:write:user` / `chat:write:bot`).
   - `chat:write.public` — required to post to public channels the app isn't a member of. **Must be requested alongside `chat:write`** — the public variant doesn't grant message-send on its own.
3. **Install to workspace** — the same page. Slack issues a **Bot User OAuth Token** (the workspace-scoped token a workflow uses). Bot tokens conventionally begin with

`xoxb-`; verify the prefix on issue and store the literal value, not the prefix family.

4. **Create the Dynatrace Connection** — *Settings* → *Integrations* → *Connections* → + *Connection* → *Slack*, paste the bot token, save under a descriptive name (e.g. `slack-prod-alerts`).

> **Slack docs:** chat.write scope (docs.slack.dev) — *"Send messages as your Slack app"*; chat.write.public scope (docs.slack.dev) — allows posting to channels the app isn't a member of, requires `chat:write` too.

Token type choice: prefer the Bot Token (OAuth app, the path above) over an Incoming Webhook URL. Webhooks are pinned to one channel and lose the ability to update / delete / thread messages, react with emoji, or use Block Kit interactivity. WFLOW-03 documents the webhook variant for the minimal case; production Slack integrations should use the OAuth app.

3.2 PagerDuty — Events API v2 Integration Key vs REST API Key

PagerDuty exposes two distinct credentials for two distinct surfaces. Picking the wrong one is the single most common WFLOW-05 setup mistake.

Credential	Where minted	What it does	Use for
Integration Key (a.k.a. routing key)	On a single PagerDuty *service* → *Integrations* tab → *Add an integration* → **Events API v2**	Sends *events* (trigger / acknowledge / resolve) to the one service it belongs to. No account-level read access.	Routing Dynatrace problems / Davis events into PagerDuty as incidents. The default WFLOW-05 happy path.
REST API Key	Account-level → *Integrations* → *API Access Keys* (general access) or *User* → *My Profile* → *User Settings* → *Create API User Token* (personal)	Account-scoped CRUD across services, schedules, escalation policies, incidents, users.	Querying on-call schedules, updating incident status from a workflow, listing services — workflow tasks that need *more than just "open an incident."*

PagerDuty's own integration guidance draws the same boundary — Events API v2 is intended for *machine-generated monitoring and event data*, REST API for *human-*

generated events, tickets or incidents requiring richer CRUD. The Dynatrace `dynatrace.pagerduty:*` action set covers both; pick the credential type matching the action's verb (trigger/ack/resolve → Integration Key; query/update incident metadata → REST API key).

dedup_key (Events API v2): each Events API payload carries an optional `dedup_key`. PagerDuty correlates subsequent `trigger`, `acknowledge`, and `resolve` events with the same `dedup_key` against the **same open incident**, so a Davis problem can drive the full lifecycle: trigger on `OPEN`, resolve on `CLOSED`, all referencing the same key. WFLOW-05 §6 already recommends `dynatrace-{problem_id}` — keep that pattern: it makes the Dynatrace problem ID the join key on both sides.

> **PagerDuty docs:** Services and integrations (PagerDuty Support) — *"Events API v2 is designed for machine-generated monitoring and event data...for human-generated events, tickets or incidents...using the REST API, which enables direct, streamlined creation of PagerDuty incidents"*; Events API v2 overview (PagerDuty Developer).

3.3 ServiceNow — OAuth Client vs Basic Auth

ServiceNow's inbound REST API accepts both **Basic Authentication** (integration-user username/password) and **OAuth 2.0** (client credentials, password, JWT bearer, or authorization-code grants). Both methods are currently supported on the platform — there is no platform-wide deprecation date for Basic Auth on inbound REST as of this writing.

The push toward OAuth is **policy-driven, not platform-driven**:

- Many enterprise security teams forbid storing plaintext integration-user passwords in third-party systems, which rules out Basic Auth even though the platform still accepts it.
- OAuth client credentials with short access-token lifetimes constrain the blast radius if a token leaks — a 30-minute access token is recoverable; an integration-user password rotated quarterly is not.

Picking the right OAuth grant:

Grant type	When to use for Dynatrace Workflows
Client Credentials	Default for workflow-to-ServiceNow. The workflow acts as itself — one client ID / secret pair per environment (prod/staging), no end-user context.

Grant type	When to use for Dynatrace Workflows
Password (Resource Owner Password Credentials)	Only when the ServiceNow integration user *must* be tracked as the actor on every record (caller_id / sys_created_by). Still stores user credentials — pick this only over Client Credentials when there's a clear audit requirement.
Authorization Code	Not applicable — workflows run unattended; there is no interactive user.

Minting an OAuth client (ServiceNow side):

1. Navigate to *System OAuth → Application Registry*.
2. **New → Create an OAuth API endpoint for external clients**.
3. Set *Name*, leave *Client ID* auto-generated, set *Client Secret* (or let SN generate). Default *Access Token Lifespan* is 30 minutes — keep short.
4. Save. Note the *Client ID* and *Client Secret* — both go into Dynatrace as connection secrets.
5. The instance URL is your own subdomain: `https://<instance>.service-now.com`. The token endpoint is `https://<instance>.service-now.com/oauth_token.do`.

Scopes in ServiceNow are *role-based on the OAuth client's linked user*, not OAuth-scope strings. Bind the OAuth client to an integration user that holds:

- `itil` and `incident_manager` (or a narrower custom role) — to read/write the `incident` table.
- `personalize_choices` if the workflow sets choice-list values that aren't in the default options.
- *Avoid* `admin`. WFLOW-05 §4 already calls out least-privilege; ServiceNow tightens it further: the integration user should only see the tables the workflow updates.

> **ServiceNow docs:** REST API authentication (ServiceNow Docs) — both OAuth 2.0 inbound and Basic Authentication remain documented; no platform-wide deprecation notice as of May 2026. Verify against your instance's release notes before assuming Basic Auth is permanent.

3.4 Rotation Patterns Without Breaking In-Flight Workflows

The instinct on rotation day is *"update the secret value and save."* That works on a quiet system; on one with running workflows it can fail mid-execution. The safer pattern is **dual-**

credential overlap:

1. **Mint the new credential** at the vendor (new Slack bot token, new PagerDuty Integration Key on the same service, new ServiceNow OAuth client). Both old and new are now active.
2. **Create a parallel Dynatrace Connection** with a `-v2` suffix (e.g. `slack-prod-alerts-v2`). Don't overwrite the existing one yet.
3. **Migrate workflows one at a time** — update each workflow's task to point at `-v2`, run an On-Demand execution, verify the result vendor-side, then move to the next workflow.
4. **Wait one full execution cycle** beyond the slowest scheduled workflow (typically 24–48 hours for daily reports; longer for weekly) to confirm no workflow still references the old connection.
5. **Revoke the old credential** at the vendor. Only after revocation is the rotation complete — if the old credential still works, the rotation has only added a credential, not retired one.
6. **Delete the old Dynatrace Connection**.

This pattern adds one day of dual-credential window, but no workflow ever runs against an invalid token. Compare with the naive "edit the secret value in place" path, which causes every in-flight task that hasn't yet sent its outbound request to fail with a 401 the moment the vendor invalidates the old value.

When to rotate:

Trigger	Cadence
Routine	Quarterly minimum for Slack bot tokens and ServiceNow OAuth secrets; annually for PagerDuty Integration Keys (lower exposure surface)
Personnel change	When the engineer who originally minted the credential leaves the team or company
Suspected exposure	Immediately. Bypass the overlap pattern; revoke first, accept the workflow outage, restore on the new credential
Vendor-forced	When Slack / PagerDuty / ServiceNow issues a security advisory or forces a token regeneration

3.5 Detecting Workflows That Reference a Specific Credential

Before rotating, find every workflow that uses the connection — both to scope the migration and to be sure step 3 of the rotation pattern is exhaustive. Workflow execution events carry the connection name in task input, so a connection-name search across recent executions is the most reliable inventory:

```
// Inventory: which workflows invoke a given connector — replace the literal
below
//
// Data object corrected 08/12/2026 along with the rest of this series:
workflow executions live in
// `dt.system.events` with `event.kind == "WORKFLOW_EVENT"`, not in `events`
under a non-existent
// `automation.task.execution` type. `task.input` is not a field either — the
connector actually
// invoked is `dt.automation_engine.action.app` / `.action.function`, which is
a better key than
// scraping an input payload for a connection name.
fetch dt.system.events, from:-30d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter contains(dt.automation_engine.action.app, "servicenow")
| summarize {
    last_execution  = takeMax(timestamp),
    execution_count = count()
  }, by:{dt.automation_engine.workflow.title, dt.automation_engine.task.name,
dt.automation_engine.action.function}
| sort last_execution desc
| limit 25
```

If a workflow shows up here, it references the connection — migrate it before revoking the old credential. If a workflow you expected to see *doesn't* show up, it either hasn't executed in 30 days (extend the lookback) or references the connection only in a code path that hasn't been hit.

For a wider sweep across all external connections:

```
// Inventory: which connector apps workflows actually invoke
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| summarize {executions = count(), workflows =
countDistinctExact(dt.automation_engine.workflow.id)}, by:
{dt.automation_engine.action.app}
| sort executions desc
| limit 25
```

Failure rates per connection type surface a soon-to-expire credential before the rotation deadline — a Slack token approaching expiry typically manifests as a rising 401 rate days before the token's nominal expiration.

3.6 Cross-References

- **WFLOW-03** (§ Setting Up Connections, § Slack Notifications, § Microsoft Teams Notifications) — minimal Connection-creation walkthrough and webhook-vs-OAuth choice for Slack/Teams.
- **WFLOW-05** (§ PagerDuty Setup, § ServiceNow Setup) — the workflow-task side: how to consume the credentials this section mints. WFLOW-05 §6 covers `dedupKey` and `correlation_id` patterns referenced above.
- **WFLOW-06** (§ Slack Block Kit, § Teams Adaptive Cards) — message-template patterns that depend on the OAuth-app credential path (Block Kit interactivity is unavailable through Incoming Webhooks).

4. Access Control (RBAC)

Workflow Permissions

Permission	Allows
`automation:workflows:read`	View workflows, view execution history
`automation:workflows:write`	Create, edit, enable/disable workflows
`automation:workflows:run`	Execute workflows manually
`automation:workflows:delete`	Delete workflows
`automation:workflows:admin`	Full admin including secrets
`automation:connections:read`	View connections
`automation:connections:write`	Create, edit connections

IAM Policy Example

```
ALLOW automation:workflows:read;
ALLOW automation:workflows:write WHERE workflow.owner == "${user.email}";
ALLOW automation:workflows:run;
```

Team-Based Access Pattern

Team	Permissions
Workflow Admins	Full admin access
SRE Team	Read, write, run all workflows
App Teams	Read, write, run own workflows
Viewers	Read-only access

Workflow Ownership

```
# Include ownership metadata in workflow
metadata:
  owner: sre-team@company.com
  team: platform
  classification: production
```

5. Workflow Observability

Key Metrics to Monitor

Metric	Query	Alert Threshold
Execution success rate	See cell below	< 95%
Execution duration	See cell below	> 5 min avg
Failed executions	See cell below	> 5 per hour
Rate limit hits	See cell below	Any

Execution History Dashboard

Build a dashboard with these queries to monitor workflow health.

```
// Overall workflow health - last 24 hours
fetch events, from: now() - 24h
| filter event.type == "automation.workflow.execution"
| summarize
    total = count(),
    succeeded = countIf(execution.status == "SUCCEEDED"),
    failed = countIf(execution.status == "FAILED"),
    timed_out = countIf(execution.status == "TIMED_OUT")
| fieldsAdd success_rate = round(100.0 * succeeded / total, decimals: 2)
```

```
// Per-workflow success rates
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {total = count(), succeeded = countIf(dt.automation_engine.state == "SUCCESS"), failed = countIf(dt.automation_engine.state == "ERROR")}, by: {dt.automation_engine.workflow.title}
| fieldsAdd success_pct = round(succeeded * 100.0 / total, decimals: 1)
| sort failed desc
| limit 25
```

```
// Execution trend over time
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| makeTimeseries executions = count(), interval:1h, by: {dt.automation_engine.state}
```

```
// Recent failures with error details
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title, dt.automation_engine.state_info, dt.automation_engine.workflow_execution.id
| sort timestamp desc
| limit 25
```

```
// Task-level performance
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {executions = count(), avg_ms = avg(duration) / 1ms}, by: {dt.automation_engine.workflow.title, dt.automation_engine.task.name}
| sort avg_ms desc
| limit 25
```

4.1 Deeper debugging patterns

The five queries above give a workflow-level health view. The patterns below drill into **per-task execution detail** — what readers actually need when a workflow execution shows

`FAILED` and the high-level dashboard doesn't reveal which task in which step broke.

Workflow Automation emits two event types in Grail:

Event type	One per	Key fields
`automation.workflow.execution`	Workflow run	`execution.id`, `execution.status`, `execution.duration`, `execution.error`, `workflow.name`, `trigger.type`
`automation.task.execution`	Task run within a workflow	`execution.id` (parent workflow run), `task.name`, `task.type`, `task.status`, `task.duration`, `task.error`

`execution.id` is the **correlation key** that joins task events to their parent workflow run, and links workflow runs to any downstream logs the run produced (HTTP-action target logs, JavaScript-action `console.log` output captured by the run, etc.).

> **Sources:** Workflows umbrella (DT docs), Workflow reference (DT docs), Dynatrace Query Language reference (DT docs).

4.2 Failed executions with task-level detail

When a workflow shows `FAILED`, the actionable question is *which task in the run failed and what was the error?* This goes one level below cell 8 (workflow-level failures) by filtering on `automation.task.execution` and surfacing `task.error`.

```
// Failed task executions in the last 24h, with their workflow
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| summarize failures = count(), by:{dt.automation_engine.workflow.title,
dt.automation_engine.task.name}
| sort failures desc
| limit 25
```

4.3 Slowest tasks across all workflows (duration percentiles)

`avg` and `max` (cell 9 above) hide long-tail behavior. A task whose p99 is 4× its p50 is a different problem from one with a uniformly high p50 — the first is a tail-latency / external-

dependency issue, the second is a structural bottleneck. Percentile aggregations make the distinction visible.

```
// Slowest tasks across all workflows (last 7d) by p95
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`, and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to `dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by `event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION · WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript, send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR · DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it is ERROR
//   .state.is_final                     true only on terminal records — filter on it, otherwise a
//                                        single execution is counted once as RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                         (was task.error)  ·  duration (was task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT" | limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {executions = count(), p95_ms = percentile(duration, 95) / 1ms}, by:{dt.automation_engine.workflow.title, dt.automation_engine.task.name}
| filter executions &gt; 5
| sort p95_ms desc
| limit 20
```

4.4 Filter by workflow ID, task name, status

Targeted lookup for a single workflow + task combination. Adapt the literals to the workflow and task you are debugging — useful when a specific notification keeps failing and you need the error history for just that step.

```
// Targeted lookup — replace the literal with a workflow title from your
tenant
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "TASK_EXECUTION"
| filter contains(dt.automation_engine.workflow.title, "Alert")
| fields timestamp, dt.automation_engine.workflow.title,
dt.automation_engine.task.name, dt.automation_engine.state, duration
| sort timestamp desc
| limit 25
```

4.5 Correlate workflow failures to downstream logs

When a workflow run emits logs (HTTP-action target service, JavaScript-action `console.log`, etc.), the run's `execution.id` is the join key. This pattern looks up logs that

contain the failing run's `execution.id` so you can read the run's downstream side-effects in one query.

> The `lookupField` in the join below assumes logs carry the workflow execution ID in the `dt.execution.id` attribute. If your downstream service writes the ID into the log body instead, swap the lookup subquery to `parse content, "...'execution_id=' DATA:exec_id"` and join on `exec_id`.

```
// Workflow failures in the last hour
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-1h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| fields timestamp, dt.automation_engine.workflow.title,
dt.automation_engine.state_info
| sort timestamp desc
| limit 25
```

4.6 Promoting these queries to a dashboard

These queries are written as ad-hoc investigation patterns — copy into a Notebook section, change the time range, run. To make them part of standing operations, promote

them to a dashboard:

- **Workflow Health overview** — cells 4.1-style success-rate tile + cell 4.3 percentile table + cell 4.2 recent-failures list.
- **Per-workflow drilldown** — parameterize cell 4.4 with a `workflow.name` variable so SREs can pick a workflow from a dropdown.
- **Cross-signal correlation** — cell 4.5 next to a Davis Problems tile filtered to the same time window, so failures and incidents appear side-by-side.

Dashboard tile selection, variable binding, and layout patterns live in the **DASH** series — see *DASH-02 (tile types)*, *DASH-04 (variables and parameterization)*, and *DASH-06 (operational dashboards)*. The DQL stays the same; only the surface (Notebook section → dashboard tile) changes.

6. Alerting on Workflows

Create a Workflow-Monitoring Workflow

Use a scheduled workflow to monitor other workflows:

```
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

export default async function({ env }) {
  // Query for failures in the last hour
  const result = await queryExecutionClient.queryExecute({
    body: {
      query: `
        fetch events, from: now() - 1h
        | filter event.type == "automation.workflow.execution"
        | filter execution.status == "FAILED"
        | summarize failures = count(), by:{workflow.name}
        | filter failures &gt;= 3
      `
    }
  });

  const failingWorkflows = result.result.records || [];

  if (failingWorkflows.length &gt; 0) {
    return {
      alert: true,
      failing_workflows: failingWorkflows,
      message: `${failingWorkflows.length} workflows with 3+ failures in the
last hour`
    };
  }

  return { alert: false };
}
```

Conditional Alert Based on Check

```
conditions:
  - name: should_alert
    expression: '{{ result("check_workflows").alert }}'

tasks:
  - name: check_workflows
    type: dynatrace.automations:run-javascript
    # ... check script above

  - name: alert_slack
    type: dynatrace.slack:message
    conditions: [should_alert]
    dependsOn: [check_workflows]
    input:
      channel: "#workflow-alerts"
      message: |
        :warning: *Workflow Health Alert*

        {{ result("check_workflows").message }}

        {% for wf in result("check_workflows").failing_workflows %}
        • {{ wf.workflow_name }}: {{ wf.failures }} failures
        {% endfor %}
```

7. Change Management

Version Control

Export workflows as JSON for version control:

```
# Export via API
curl -X GET "https://&lt;env&gt;/platform/automation/v1/workflows/&lt;id&gt;"
\
  -H "Authorization: Api-Token &lt;token&gt;" \
  -o workflow-backup.json
```

Workflow Naming Convention

```
&lt;team&gt;-&lt;function&gt;-&lt;environment&gt;

Examples:
- sre-problem-notifications-prod
- checkout-auto-remediation-staging
- platform-daily-report-all
```

Testing Changes

1. **Clone workflow** to test version
2. **Modify clone** with changes
3. **Test with On-Demand trigger**
4. **Review execution results**
5. **Promote to production workflow**

Rollback Procedure

1. Disable failing workflow (toggle off)
2. Import previous version from backup
3. Enable restored workflow
4. Verify execution

8. Operational Runbook

Daily Operations

Task	Frequency	Action
Check dashboard	Daily	Review execution success rates
Review failures	Daily	Investigate and fix any failures
Verify connections	Weekly	Test all external connections
Rotate secrets	Monthly	Update API tokens/passwords

Troubleshooting Common Issues

Symptom	Likely Cause	Resolution
Task timeout	External API slow	Increase timeout, add retry
Auth failure	Expired token	Rotate secret
Rate limit	Too many executions	Add throttling, check trigger
Missing data	Query returned empty	Check time range, filters

Escalation Path

```
Level 1: Check execution history, review error message
Level 2: Check connections, verify external services
Level 3: Review workflow logic, check for regressions
Level 4: Contact Dynatrace support
```

Capacity Planning

Limit	Value	Monitor
Max concurrent executions	100	Dashboard query
Max execution time	15 min	Duration metrics
Max tasks per workflow	50	Workflow design
Rate limits	Varies	Rate limit errors

9. Series Summary

What You've Learned

Notebook	Key Topics
WFLOW-01	Workflow fundamentals, components, first workflow
WFLOW-02	Trigger types: detected problem, metric, schedule, event

Notebook	Key Topics
WFLOW-03	Slack, Teams, email notification basics
WFLOW-04	Conditional routing, escalation patterns
WFLOW-05	PagerDuty and ServiceNow integration
WFLOW-06	Custom templates, Jinja, Block Kit, Adaptive Cards
WFLOW-07	Auto-remediation, guardrails, approval workflows
WFLOW-08	JavaScript SDK, HTTP requests, custom integrations
WFLOW-09	Security, governance, monitoring, operations

Implementation Checklist

- [ ] Create connections for notification channels
- [ ] Build problem notification workflow
- [ ] Configure severity-based routing
- [ ] Integrate incident management (PagerDuty/ServiceNow)
- [ ] Design custom notification templates
- [ ] Implement auto-remediation (if applicable)
- [ ] Set up workflow monitoring dashboard
- [ ] Configure workflow health alerts
- [ ] Document operational procedures
- [ ] Train team on workflow management

Best Practices Summary

1. **Start simple** - Basic notifications before complex automation
2. **Test thoroughly** - Use On-Demand trigger for testing
3. **Secure by default** - Never hardcode secrets
4. **Monitor everything** - Dashboard for workflow health
5. **Document changes** - Version control and change log
6. **Plan for failure** - Error handling and rollback

Congratulations!

You've completed the WFLOW series. You now have the knowledge to:

- Build event-driven notification workflows
- Integrate with incident management platforms
- Create custom automation with JavaScript
- Implement auto-remediation safely
- Operate workflows in production

References

- Workflows umbrella (DT docs)
- Workflow actions umbrella (DT docs)
- Workflow reference / Jinja expressions (DT docs)
- Identity and Access Management umbrella (DT docs)
- Manage user permissions / boundaries (DT docs)
- Dynatrace Developer Portal (Dynatrace)
- Slack chat.write scope (docs.slack.dev)
- Slack chat.write.public scope (docs.slack.dev)
- PagerDuty Services and integrations (PagerDuty Support)
- PagerDuty Events API v2 overview (PagerDuty Developer)
- ServiceNow inbound REST API (ServiceNow Docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.